

# Trabalho Prático 2 - Sistema de Despacho de Transporte por Aplicativo

Henrique do Vale Ribeiro

Universidade Federal de Minas Gerais (UFMG)  
Belo Horizonte - MG - Brasil

henriquevrhb@gmail.com

## 1 Introdução

A crescente demanda por soluções de mobilidade urbana eficientes e de baixo custo impulsiona a inovação no setor de transporte por aplicativo. Nesse contexto, a empresa Cabe Aí buscou integrar um serviço de corrida compartilhada, similar ao táxi-lotação, como estratégia de otimização de recursos e redução de custos para o usuário.

O objetivo principal deste trabalho consiste no desenvolvimento de um **Sistema de Despacho de Corridas** capaz de, diante de uma nova demanda, não apenas alocá-la a um veículo, mas também avaliar e propor combinações com corridas já existentes.

O desafio algorítmico central reside em identificar grupos de demandas que satisfaçam simultaneamente um conjunto rigoroso de critérios: a **capacidade do veículo** ( $\eta$ ), as **restrições temporais** (janela  $\delta$ ), os limites de **desvio geométrico** ( $\alpha$  e  $\beta$ ) e, sobretudo, uma **métrica de eficiência** ( $\lambda$ ) que assegura a viabilidade econômica da rota compartilhada.

A solução adotada utiliza a metodologia de Simulação de Eventos Discretos (SED), dividida em duas fases principais:

1. **Fase de Matching e Construção de Rota:** leitura e agrupamento das demandas em corridas compartilhadas, respeitando integralmente os critérios de elegibilidade.
2. **Fase de Simulação:** processamento cronológico dos eventos de embarque, desembarque e finalização da corrida.

Este documento descreve detalhadamente a solução desenvolvida e os resultados obtidos. A seção Método apresenta a arquitetura do software, os TADs customizados e os algoritmos de matching e simulação. Em seguida, são discutidas a Análise de Complexidade e as Estratégias de Robustez implementadas. Por fim, a Análise Experimental avalia o desempenho do sistema sob diferentes parametrizações, validando sua eficácia e viabilidade operacional.

## 2 Método

O programa foi implementado na linguagem C++11 e compilado pelo compilador G++ (GNU Compiler Collection). A máquina utilizada possui um Sistema Operacional Linux (Ubuntu 24.04.2 LTS), processador Intel(R) Core(TM) i7-6700 CPU @ 3.40GHz e 16GB de RAM.

## 2.1 Estruturas de Dados e Algoritmos Fundamentais

### 2.1.1 ListaArranjo (Vetor Dinâmico)

A classe **ListaArranjo** consiste em uma lista linear com alocação sequencial, implementada como template genérico. Ela serve como contêiner principal do sistema, substituindo os vetores da STL conforme exigido pelo projeto.

**Gerenciamento de Memória:** A estrutura mantém internamente um ponteiro bruto e duas variáveis de controle: **capacidade** (memória física alocada) e **tamanho** (elementos válidos). Diferente de arranjos estáticos, implementa redimensionamento automático.

**Estratégia de Expansão:** Quando a capacidade é atingida, a estrutura realiza expansão geométrica (duplicando a capacidade). O processo envolve alocação de novo bloco, cópia dos dados e liberação do bloco anterior. Essa estratégia assegura custo de inserção  $O(1)$  amortizado, adaptando-se ao crescimento natural da simulação.

### 2.1.2 MinHeap (Fila de Prioridade)

A classe **MinHeap** implementa uma fila de prioridade mínima sobre uma árvore binária completa. Ela atua como motor da simulação, garantindo acesso ao próximo evento em  $O(1)$ .

**Representação Implícita:** A estrutura utiliza a **ListaArranjo** como vetor subjacente, mapeando a topologia da árvore diretamente nos índices, o que favorece a localidade de referência. Para um nó em  $i$ , o pai está em  $\lfloor (i-1)/2 \rfloor$  e os filhos em  $2i+1$  e  $2i+2$ .

**Propriedade de Heap:** A invariante que garante que o pai possui chave menor ou igual à dos filhos é mantida pelos algoritmos **heapifyUp** (durante inserção, fazendo o elemento subir) e **heapifyDown** (durante remoção, afundando a nova raiz). Ambos asseguram a consistência estrutural a cada operação.

## 2.2 Tipos Abstratos de Dados (TADs)

Para representar com precisão as entidades do problema e os resultados produzidos, foram modelados os seguintes Tipos Abstratos de Dados (TADs), cada um encapsulando dados e funções associados.

### 2.2.1 Entidades Básicas de Domínio

Estas classes representam os elementos fundamentais da geografia e da logística, atuando como agregadores de dados com métodos auxiliares de cálculo.

- **TAD Ponto:** Representa uma coordenada cartesiana  $(x, y)$  e encapsula o cálculo da distância euclidiana entre pontos.
- **TAD Demanda:** Modela uma solicitação de transporte, armazenando identificador, tempo de chamada e pontos de origem e destino. Mantém também o estado da solicitação (ex.: **DEMANDADA**, **COMPARTILHADA**), atualizado conforme o avanço da simulação.
- **TAD Parada:** Representa um ponto de parada na rota, contendo coordenadas, tipo da operação (**EMBARQUE** ou **DESEMBARQUE**) e o identificador da demanda associada.
- **TAD Trecho:** Segmento entre duas paradas consecutivas. Armazena ponteiros para as paradas inicial e final, o tipo do trecho (coleta, entrega ou deslocamento) e pré-calcula distância e tempo de percurso para maior eficiência.

## 2.2.2 Componentes de Simulação

Estas classes organizam o fluxo temporal da Simulação de Eventos Discretos (SED), abstraindo a complexidade da ordenação.

- **TAD Evento:** Unidade atômica da simulação. Cada evento armazena tempo (chave de ordenação), tipo (ex.: chegada em embarque) e ponteiro para a corrida correspondente. Operadores de comparação garantem sua ordenação correta no MinHeap.
- **TAD Escalonador:** Interface pública para o motor de simulação. Encapsula o MinHeap de eventos, oferecendo métodos de inserção e remoção que isolam a lógica temporal da lógica de negócio.

## 2.2.3 Gerenciador de Rotas (Corrida)

A classe **Corrida** é o núcleo da lógica de negócios, responsável por agrupar demandas e gerenciar a rota compartilhada.

- **TAD Corrida:** Mantém listas dinâmicas de demandas, paradas e trechos. Implementa a geração da rota básica, o cálculo da métrica de eficiência e a lógica de transição de estados da simulação, retornando o próximo evento ao controlador principal.

## 2.3 Principais Algoritmos e Funções

A lógica do sistema emerge da interação entre os TADs descritos. A seguir, resumem-se os métodos e algoritmos essenciais ao processo de matching e simulação.

### 2.3.1 Algoritmos da Classe Corrida

- **checaDistancias:** Verifica as restrições geométricas  $\alpha$  e  $\beta$ , percorrendo todas as demandas já alocadas e calculando distâncias entre origens e destinos. Caso algum limite seja excedido, retorna **false** imediatamente.
- **gerarRotaBasica:** Constrói a rota compartilhada seguindo a estratégia embarques  $\rightarrow$  desembarques:
  1. Limpa rota anterior.
  2. Cria paradas de **EMBARQUE** e, em seguida, de **DESEMBARQUE**.
  3. Conecta paradas consecutivas com Trechos, inferindo o tipo automaticamente.

- **calcularEficiencia:** Calcula a métrica:

$$E = \frac{\sum D_{individual}}{\sum D_{trechos}}$$

Avalia se a rota compartilhada apresenta eficiência mínima aceitável; se  $E < \lambda$ , a combinação é descartada.

- **processarEvento:** Atualiza o estado temporal da corrida. Dado um evento na parada  $i$ :
  - Se  $i$  for a última parada, retorna **FIM\_DA\_CORRIDA**.
  - Caso contrário, identifica o trecho seguinte, utiliza o tempo pré-calculado e agenda o evento correspondente à próxima parada.

### 2.3.2 Algoritmo de Construção (Matching)

Implementado em **construirCorridas()**, utiliza uma abordagem gulosa para formar corridas. Para cada demanda base  $c_0$ , uma nova corrida é criada e tenta-se adicionar demandas  $c_i$  que satisfaçam:

1. **Temporal:**  $|c_i - c_0| < \delta$ .
2. **Capacidade:** total de passageiros  $\leq \eta$ .
3. **Geométrica:** **checaDistancias** retorna **true**.
4. **Eficiência:** após inserção provisória, **calcularEficiencia**  $\geq \lambda$ .

### 2.3.3 Algoritmo de Simulação (Fase 2)

A função **executarSimulacao** processa todos os eventos:

1. Remove o próximo evento do **MinHeap**.
2. Encaminha-o à **Corrida** correspondente.
3. Eventos intermediários geram novos eventos que retornam ao escalonador.
4. Eventos de término acionam a impressão das estatísticas finais.

## 3 Análise de Complexidade

Esta seção apresenta a análise de complexidade assintótica dos algoritmos e procedimentos implementados, avaliando tanto a complexidade de tempo, quanto a de espaço. Funções auxiliares de custo constante trivial, como getters e setters, são omitidas.

Para fins de análise, definem-se as variáveis:

- $D$ : Número total de demandas no arquivo de entrada.
- $E$ : Número total de eventos processados na simulação (onde  $E \approx 2D$ , pois cada demanda gera embarque e desembarque).
- $\eta$ : Capacidade máxima do veículo (constante pequena).
- $M$ : Tamanho atual das estruturas de dados (vetor ou heap).

### 3.1 Complexidade das Estruturas de Dados

A eficiência do sistema fundamenta-se nas operações das estruturas customizadas.

- **ListaArranjo:**
  - **Tempo:** Acesso em  $O(1)$ . Inserção (**push.back**) em  $O(1)$  *amortizado*. A expansão ocorre apenas quando a capacidade é atingida, com custo  $O(M)$ , mas sua frequência logarítmica dilui o custo por operação.
  - **Espaço:**  $O(M)$ . O uso de memória é contíguo e proporcional ao número de elementos armazenados.
- **MinHeap:**
  - **Tempo:** Tanto a inserção (**push**) quanto a remoção da raiz (**pop**) têm complexidade  $O(\log M)$ , correspondente à altura da árvore binária implícita percorrida pelas funções **heapifyUp** e **heapifyDown**.
  - **Espaço:**  $O(M)$  para o armazenamento do vetor. Adicionalmente, a implementação recursiva do **heapifyDown** utiliza  $O(\log M)$  de espaço na pilha de execução (**stack**) no pior caso.

## 3.2 Complexidade dos Procedimentos da Corrida

Os métodos da classe Corrida (`checaDistancias()`, `gerarRotaBasica()`, `calcularEficiencia()`) iteram sobre as demandas ou paradas já alocadas na corrida.

Como o número máximo de passageiros é limitado pela capacidade do veículo ( $\eta$ ), o número de iterações é limitado por  $\eta$ . Considerando que  $\eta$  é uma constante de configuração (geralmente pequena) e não cresce com a entrada  $D$ , a complexidade assintótica destes métodos é:

$$C_{corrida} = O(\eta) \approx \mathbf{O(1)}$$

Isto é crucial para garantir que as verificações internas não degradem a performance do algoritmo de matching.

## 3.3 Complexidade dos Algoritmos Principais

### 3.3.1 Fase 1: Matching e Construção (Algoritmo Guloso)

Esta fase é composta por dois laços aninhados que percorrem a lista de demandas para formar as corridas.

- **Laço Externo:** Itera sobre as demandas ainda não alocadas. Pior caso:  $O(D)$ .
- **Laço Interno:** Busca candidatos para agrupar com a demanda atual. Pior caso:  $O(D)$ .
- **Operações Internas:** Dentro do laço interno, métodos críticos da classe Corrida são chamados:
  - `checaDistancias()` e `adicionarDemanda()`:  $O(1)$  ou  $O(\eta)$ .
  - `gerarRotaBasica()`: Recalcula a rota a cada tentativa. Dependendo da implementação de ordenação das paradas, o custo é  $O(\eta^2)$  ou  $O(\eta \log \eta)$ .
  - `calcularEficiencia()`: Percorre as paradas da rota:  $O(\eta)$ .

A complexidade de tempo da Fase 1 é dada pelo produto dos laços e do custo interno:

$$C_{matching}(D) = O(D \times D \times \eta^2) = O(D^2 \eta^2)$$

Considerando que a capacidade do veículo  $\eta$  é uma constante pequena e fixa em relação à entrada  $D$  (ou seja,  $\eta \ll D$ ), o termo  $\eta^2$  é absorvido na constante assintótica:

$$\mathbf{C_{matching}(D) = O(D^2)}$$

### 3.3.2 Fase 2: Simulação de Eventos

O motor de simulação processa a fila de prioridade (MinHeap) contendo os eventos discretos.

- **Número de Iterações:** O loop `while` processa cada evento de embarque e desembarque, totalizando  $E$  iterações.
- **Operações de Heap:** Retirar o próximo evento e inserir novos eventos custa  $O(\log N_{heap})$ . Como há no máximo um evento futuro por corrida ativa,  $N_{heap} \leq D$ , logo o custo é  $O(\log D)$ .
- **Processamento da Corrida:** O método `processarEvento()` e a impressão final dependem do número de passageiros atuais, custando  $O(\eta)$ .

A complexidade da simulação é:

$$C_{simulacao}(E) = O(E \times (\log D + \eta))$$

Dado que  $E \approx 2D$  e  $\eta$  é constante:

$$\mathbf{C_{simulacao}(D) = O(D \log D)}$$

### 3.3.3 Complexidade de Tempo Total

A complexidade total do sistema é a soma das duas fases. Como o termo quadrático domina o termo log-linear:

$$C_{total}(D) = O(D^2) + O(D \log D) = \mathbf{O(D^2)}$$

### 3.4 Complexidade de Espaço Total

A complexidade de espaço é linear em relação à entrada.

- **Armazenamento:** As listas de demandas e as estruturas internas das corridas ocupam  $O(D)$ .
- **Fila de Eventos:** O MinHeap armazena, no máximo, um evento futuro por corrida ativa simultaneamente, sendo limitado superiormente por  $O(D)$ .

Conclui-se que a complexidade de espaço total é  $\mathbf{O(D)}$ .

## 4 Estratégias de Robustez

Para assegurar a estabilidade do sistema e prevenir comportamentos indefinidos durante a execução, foram adotadas as seguintes práticas de programação defensiva:

### 4.1 Validação de Entrada e Fontes de Dados

O sistema utiliza `std::istream` para unificar o tratamento da entrada, permitindo ler tanto de arquivos quanto da entrada padrão. Caso um arquivo seja informado via linha de comando, o programa tenta abri-lo e valida sua integridade com `is_open()`, emitindo mensagens de erro claras e encerrando a execução de forma segura quando necessário. Na ausência de argumentos, o sistema recorre automaticamente ao `std::cin`, garantindo compatibilidade com ambientes de teste automatizado e uso interativo. Essa abordagem assegura robustez no processo de leitura e previne falhas decorrentes de fontes de dados inválidas.

### 4.2 Gerenciamento Centralizado de Memória:

Devido à restrição de não utilização de "smart pointers" da biblioteca padrão, implementou-se um controle manual rigoroso. A função auxiliar `limparMemoria()` atua como um "garbage collector" determinístico ao final da execução, iterando sobre as listas de **Demanda** e **Corrida** para desalocar cada objeto individualmente antes de liberar os vetores, garantindo a ausência de vazamentos de memória (memory leaks).

### 4.3 Integridade das Estruturas de Dados:

As estruturas customizadas possuem proteções internas. A **ListaArranjo** verifica limites de capacidade antes de inserções, expandindo-se automaticamente para evitar "buffer overflows". Analogamente, o **MinHeap** e o laço principal da simulação garantem que operações de remoção (pop) jamais sejam chamadas em uma estrutura vazia, prevenindo falhas de segmentação.

## 5 Análise Experimental

Esta seção apresenta a análise experimental, que investiga a sensibilidade do algoritmo de despacho em relação aos seus parâmetros restritivos. O objetivo é compreender o compromisso (trade-off) entre

o relaxamento dos filtros de agrupamento e o desempenho do sistema, avaliado sob duas métricas principais: o **Tempo de Execução** (custo computacional) e o **Número de Corridas Compartilhadas** (eficácia do serviço).

A metodologia para a condução dos experimentos foi automatizada para garantir a consistência e a reprodutibilidade dos resultados. Primeiramente, foi desenvolvido um script gerador em Python para criar um cenário de teste realista, contendo 300 demandas distribuídas em aglomerados geográficos e temporais, simulando um horário de pico urbano. A execução dos testes foi orquestrada por um segundo script de automação, responsável por variar os parâmetros de entrada, executar o binário C++ e cronometrar o tempo de processamento de cada iteração. Os dados coletados foram processados e a biblioteca matplotlib do Python foi utilizada para gerar os gráficos comparativos apresentados a seguir.

A análise foi dividida em quatro experimentos distintos, cada um focado em isolar uma dimensão crítica do algoritmo de matching:

- **Impacto da Janela Temporal ( $\delta$ ):** Variou-se  $\delta$  para analisar como a tolerância temporal afeta o custo computacional, devido ao aumento de candidatos no laço interno, e a capacidade de encontrar combinações válidas.
- **Impacto das Restrições Geométricas ( $\alpha$  e  $\beta$ ):** Avaliou-se a sensibilidade ao relaxamento dos limites de distância, mensurando o crescimento da taxa de compartilhamento com desvios maiores e seu impacto no tempo de busca.
- **Impacto da Eficiência ( $\lambda$ ):** Investigou-se a variação da exigência econômica para identificar o ponto de corte onde rotas, embora geometricamente viáveis, tornam-se inviáveis e são rejeitadas pelo filtro.
- **Impacto da Capacidade do Veículo ( $\eta$ ):** Testou-se a variação da lotação máxima para identificar o ponto de saturação do compartilhamento, determinando se o gargalo é o espaço físico ou as restrições de qualidade dos demais parâmetros.

## 5.1 Resultados: Impacto da Janela Temporal ( $\delta$ )

A Figura 1 apresenta o comportamento do sistema ao variar a janela de tempo  $\delta$  de 0 a 2400 segundos, mantendo os demais parâmetros fixos.

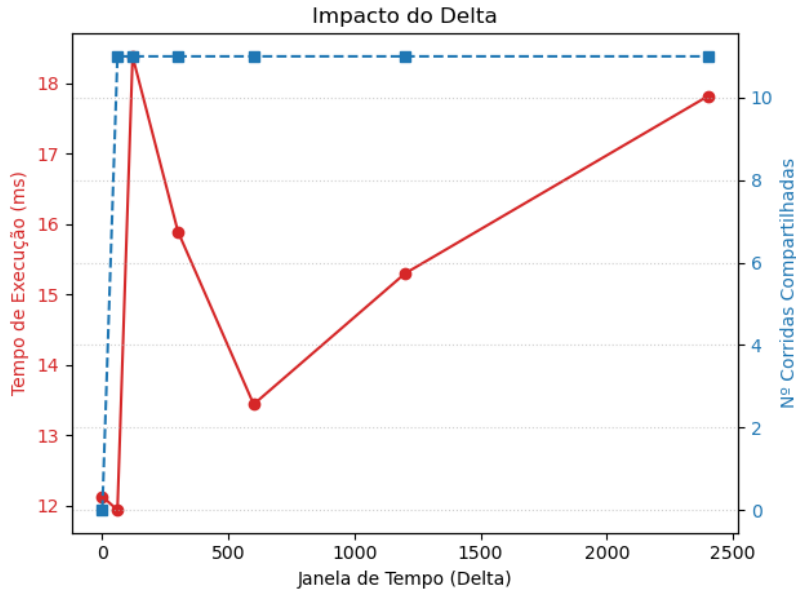


Figura 1: Impacto da variação de  $\delta$  no tempo de execução e no número de compartilhamentos.

Observa-se que o tempo de execução (linha vermelha) apresenta uma tendência de crescimento conforme  $\delta$  aumenta. Isso corrobora a análise de complexidade, pois uma janela temporal maior aumenta o número de iterações no laço interno do algoritmo de matching.

Em relação à eficácia (linha azul), nota-se um crescimento inicial rápido no número de corridas compartilhadas, que logo apresenta uma estabilização. Isso indica que, após um certo ponto, aumentar a janela de espera não gera novos agrupamentos válidos, pois os candidatos passam a ser rejeitados pelos filtros subsequentes (geométrico ou de eficiência).

## 5.2 Resultados: Impacto das Restrições Geométricas ( $\alpha$ e $\beta$ )

A Figura 2 ilustra a sensibilidade do algoritmo ao relaxamento das distâncias máximas de desvio ( $\alpha$  e  $\beta$ ), variando de 100m a 5000m.

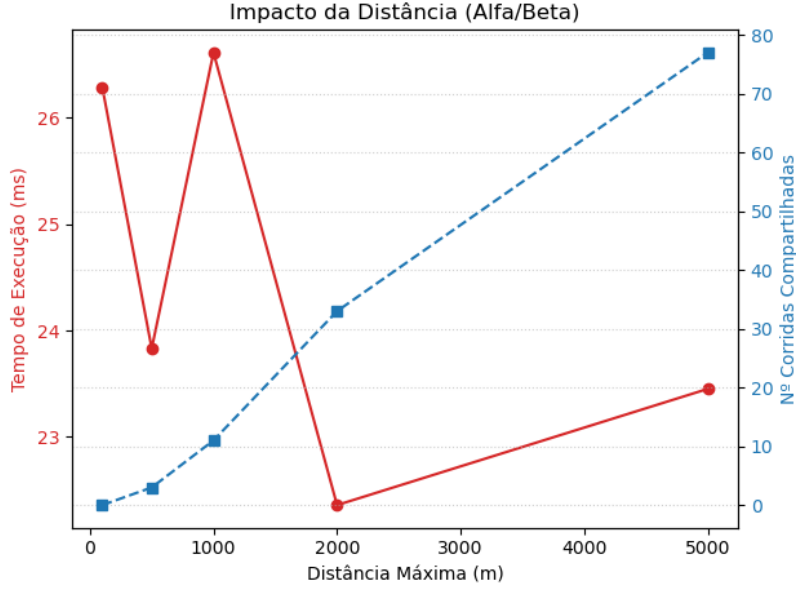


Figura 2: Impacto da variação das distâncias  $\alpha$  e  $\beta$  no desempenho.

Os resultados mostram uma forte correlação linear entre o relaxamento das restrições geográficas e o número de corridas compartilhadas (linha azul). Ao ampliar a área de busca ( $\alpha$  e  $\beta$ ), o sistema passa a capturar mais demandas vizinhas, indicando que a distribuição espacial favorece o agrupamento quando os limites de distância são expandidos.

Em contraste, o tempo de execução (linha vermelha) apresenta oscilações próprias da natureza heurística do algoritmo de matching. Enquanto  $\delta$  expande linearmente o espaço de busca temporal,  $\alpha$  e  $\beta$  funcionam como filtros de corte espacial, e essas oscilações refletem a topologia dos dados: em certos valores (ex.: 2000m), múltiplos clusters espaciais tornam-se conectados, permitindo encontrar combinações viáveis rapidamente e reduzindo o retrabalho na busca; já em faixas intermediárias (ex.: 1000m), o algoritmo consome mais tempo avaliando candidatos que passam pelo filtro geométrico, mas falham posteriormente no critério de eficiência ( $\lambda$ ). Por fim, o custo constante  $O(1)$  da verificação de distâncias justifica a ausência de tendência linear de crescimento no tempo de execução, em contraste com o impacto direto do aumento da janela temporal  $\delta$  na complexidade do algoritmo.

## 5.3 Resultados: Impacto da Eficiência ( $\lambda$ )

A Figura 3 analisa o efeito do filtro econômico, variando o limiar  $\lambda$  de 0.1 (permissivo) a 0.9 (rigoroso).



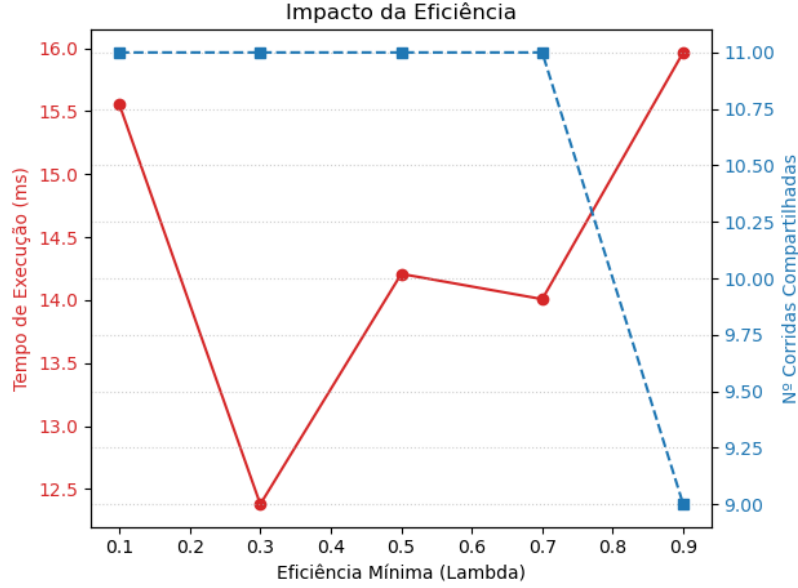


Figura 3: Impacto da variação da eficiência mínima  $\lambda$ .

O gráfico corrobora o papel do  $\lambda$  como filtro de qualidade. A taxa de compartilhamento (linha azul) permanece estável para valores baixos e médios, mas cai abruptamente próximo a  $\lambda = 0.9$ . Isso ocorre porque a métrica de eficiência ( $E = \sum D_{ind}/D_{total}$ ) tende a ser permissiva para pequenos desvios (valores  $> 1.0$  são comuns), mas torna-se restritiva para desvios significativos quando o limiar é elevado. A queda confirma que o algoritmo rejeita rotas com penalidades de deslocamento excessivas, validando a eficácia do filtro econômico na manutenção do nível de serviço.

#### 5.4 Resultados: Impacto da Capacidade do Veículo ( $\eta$ )

Para avaliar o limite físico do compartilhamento, a Figura 4 apresenta o comportamento do sistema variando a capacidade do veículo  $\eta$  de 1 (táxi individual) a 10 (van), mantendo a eficiência  $\lambda = 0.4$ .

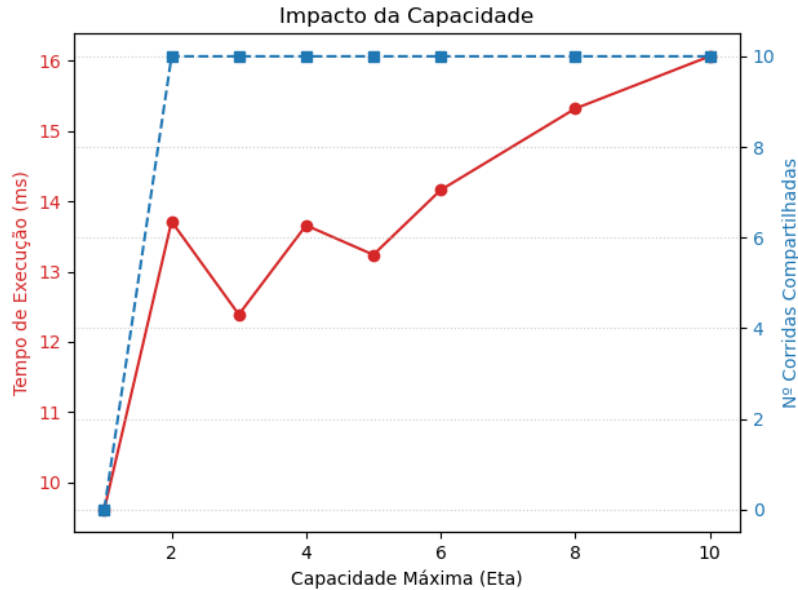


Figura 4: Impacto da variação da capacidade máxima  $\eta$  no desempenho.

O gráfico revela um fenômeno interessante de saturação. Quando  $\eta = 1$ , o número de compartilhamentos é zero, como esperado. Ao aumentar para  $\eta = 2$ , o sistema atinge imediatamente o seu potencial máximo de agrupamento para o cenário testado. Aumentos subsequentes na capacidade ( $\eta > 2$ ) não resultam em mais passageiros por corrida, indicando que as restrições de eficiência e tempo são os fatores limitantes reais, e não o espaço físico no veículo.

Por outro lado, o tempo de execução (linha vermelha) apresenta uma tendência de crescimento. Isso demonstra que, mesmo sem sucesso no agrupamento adicional, o algoritmo consome recursos computacionais tentando preencher a capacidade extra disponível, evidenciando o custo do processamento de matching em veículos maiores.

## 6 Conclusões

Neste trabalho, desenvolveu-se um **Sistema de Despacho de Transporte por Aplicativo** baseado em Simulação de Eventos Discretos (SED) e em estruturas de dados implementadas manualmente (**ListaArranjo** e **MinHeap**), em conformidade com as restrições propostas e como alternativa eficiente às bibliotecas da STL.

A análise experimental evidenciou de maneira clara as relações de compromisso que regem o processo de compartilhamento. O relaxamento das restrições geométricas ( $\alpha$  e  $\beta$ ) mostrou-se o principal fator de aumento da taxa de agrupamento, enquanto o parâmetro de eficiência ( $\lambda$ ) atuou como o filtro mais rigoroso da qualidade do serviço, reduzindo drasticamente os agrupamentos quando exigência econômica é elevada. Observou-se ainda que a capacidade do veículo ( $\eta$ ) exerce impacto marginal na taxa de compartilhamento: a limitação prática não está no número de assentos, mas sim nos critérios temporais, geométricos e econômicos que moldam as oportunidades de combinação.

Do ponto de vista computacional, os experimentos confirmaram integralmente a análise assintótica. A etapa de matching revelou-se o gargalo dominante, com comportamento quadrático  $O(D^2)$  particularmente sensível ao aumento da janela temporal ( $\delta$ ), que amplia o número de candidatos no laço interno. Em contraste, a fase de simulação manteve-se altamente eficiente, sustentada pela estrutura de prioridade (MinHeap) e operando com desempenho compatível com  $O(E \log E)$ .

Assim, além de cumprir as exigências de gerenciamento manual de memória, o projeto demonstrou na prática como escolhas de estruturas de dados, mecanismos de filtragem e parâmetros de negócio influenciam diretamente a escalabilidade do sistema. A arquitetura consolidada mostrou-se robusta, eficiente e capaz de modelar com precisão cenários urbanos complexos, validando tanto a implementação quanto os fundamentos teóricos estudados na disciplina.

## 7 Referências

MEIRA JR., Wagner; LACERDA, Anísio. Slides virtuais da disciplina de Estruturas de Dados. Belo Horizonte: Departamento de Ciência da Computação, Universidade Federal de Minas Gerais, 2025. Disponibilizado via Moodle.

CHAIMOWICZ, Luiz. Vídeo-aulas da disciplina de Estruturas de Dados. Belo Horizonte: Departamento de Ciência da Computação, Universidade Federal de Minas Gerais, 2025. Disponibilizado via Moodle.